PROJETO EVOLUTIVO DE DADOS / ENTREGA PARCIAL

Programa Vila Viva

Automação com Power Query

Como as regras das planilhas manuais passaram a gerar resultados atualizáveis por município e ano para as três doenças.

ETAPA 02 / MICROINFORMÁTICA APLICADA (ILP590)

Fatec São Paulo · Projeto orientado pelo professor Veríssimo

COVID-19 | Dengue | Pneumonia

IDENTIFICAÇÃO DA EQUIPE A CONFIRMAR

Três origens, três resultados

As mesmas planilhas -1 sustentam o tratamento manual documentado e a atualização no Power Query.

Base	Versão usada como origem	Fluxo ou destino	Outra versão localizada
COVID-19	Vila_Viva_COVID19-1.xlsx	COVID19!A4 · tbCOVID19	Vila_Viva_COVID19_Atualizado.xlsx
Dengue	Vila_Viva_Dengue-1.xlsx	Dengue!A4 · tbDengue	Vila_Viva_Dengue.xlsx
Pneumonia	Vila_Viva_Pneumonia-1.xlsx	Pneumonia!A4 · tbPneumonia	Vila_Viva_Pneumonia_Atualizado.xlsx

Chave de leitura dos resultados / Ano + Codigo_Municipio_CVE

Cada origem contém 2.580 registros de município e ano. São 645 municípios em cada ano de 2022 a 2025. Os nomes das abas nos arquivos são Bruto, Tratado, Cadastros e Auditoria.

O projeto busca apoiar o monitoramento territorial em saúde. A chave comum permite comparar a cobertura por município e ano, mantendo separados os indicadores de notificações, casos, óbitos e internações.

As consultas e os parâmetros estão em Vila_Viva_02_PowerQuery.xlsx. Os rascunhos Power_Query/COVID19.m, Power_Query/Dengue.m e Power_Query/Pneumonia.m são anteriores. O apêndice usa o código incorporado no Excel.

Ordem das etapas

As três consultas seguem o mesmo encadeamento. A função Calcular muda conforme a doença.

Ordem	Etapas e funções	O que ocorre
01	Caminho · Arquivo · Aba	Lê o parâmetro, abre o .xlsx com Excel.Workbook e seleciona a planilha de origem sem depender de maiúsculas ou minúsculas.
02	Bruto · Cabecalhos · TiparChaves · SoRegistros	Promove os cabeçalhos, tipa ano e código CVE em pt-BR e mantém linhas com as duas chaves preenchidas.
03	Unicas · CadastroMunicipal · JuncaoCadastro	Verifica unicidade do Bruto e do cadastro. Localiza o cabeçalho de Cadastros e faz junção à esquerda por código CVE.
04	CamposReferencia · Referencia · JuncaoReferencia · Dados	Seleciona campos fixos de Tratado e junta por Ano + Codigo_Municipio_CVE. A aba Auditoria serve como evidência, sem alimentar os cálculos.
05	Calcular · Calculadas · ComTipos	Executa cálculos e checagens por registro, monta as colunas na ordem do Tratado e aplica tipos às colunas definidas.
06	Tabela vinculada no Excel	Carrega as três consultas em planilhas separadas, com cabeçalho na linha 4. Os parâmetros não geram tabelas.

COVID-19

A consulta COVID19 recalcula casos, óbitos, incidência, mortalidade e letalidade.

pArquivoCOVID19 → Bruto → COVID19 → COVID19!A4 / tbCOVID19

Cadastros fornece código IBGE e nome. Tratado fornece ID_Registro, Fonte, Status_IBGE, Data_Extracao, Codigo_IBGE_7_Digitos e Municipio_IBGE_2025. A população é lida de Bruto[Populacao_Fonte].

Regra manual no Excel	Consulta e etapa no Power Query	Resultado da comparação
COUNT dos 24 meses e somas de casos e óbitos	COVID19 · Calcular: TemNumero, SomaDireta	2.580 linhas iguais. Adamantina 2022: 6.959 casos e 38 óbitos.
Incidência, mortalidade e letalidade	COVID19 · Calcular: Razao	Taxas iguais na base comparada. A letalidade permanece em escala de 0 a 100.
Checagens de fonte em G:H	COVID19 · Calcular: checagemTaxasFonte, checagemObitosFonte	2.580 OK em cada campo. Nas 36 fórmulas deslocadas de H, a consulta usa a mesma chave da linha.
Busca, limpeza de texto e status	COVID19 · JuncaoCadastro, JuncaoReferencia, Calcular	Resultados iguais aos salvos. O status DADOS A CONFERIR continua nas 2.580 linhas.

A tabela de saída é carregada pela consulta. Nenhuma coluna calculada depende de fórmulas do Excel no arquivo automatizado.

Dengue

A consulta Dengue mantém o recorte anual e distingue notificações de confirmações.

pArquivoDengue → Bruto → Dengue → Dengue!A4 / tbDengue

Cadastros fornece código IBGE e nome. Tratado fornece os seis campos de referência descritos na página de COVID-19 e Populacao_Residente. A coluna opcional Populacao_Residente de Bruto tem prioridade se preenchida.

Regra manual no Excel	Consulta e etapa no Power Query	Resultado da comparação
Referências AV:AW e soma de confirmados	Dengue · Calcular: ReferenciaDireta, Aritmetica	2.580 linhas iguais. Adamantina 2024: 169 confirmados.
Soma dos 12 meses de notificações	Dengue · Calcular: ValoresMensais, SomaDireta	2.580 linhas iguais. Adamantina 2024: 1.111 notificações.
IFERROR nas proporções e na incidência	Dengue · Calcular: Razao, try … otherwise	Resultados iguais. As proporções mantêm valores de 0 a 1.
Cadastro, checagens e campos fixos	Dengue · JuncaoCadastro, JuncaoReferencia, Calcular	Checagens iguais. IDs e texto de Data_Extracao são mantidos nas chaves existentes.

A tabela de saída é carregada pela consulta. Nenhuma coluna calculada depende de fórmulas do Excel no arquivo automatizado.

Pneumonia

A consulta Pneumonia recalcula internações, validação da soma e taxa por 100 mil habitantes.

pArquivoPneumonia → Bruto → Pneumonia → Pneumonia!A4 / tbPneumonia

Cadastros fornece código IBGE e nome. Tratado fornece Populacao_Residente por município e ano. Se preenchida, a coluna opcional Populacao_Residente de Bruto tem prioridade. A descrição da fonte segue o texto fixo do original.

Regra manual no Excel	Consulta e etapa no Power Query	Resultado da comparação
COUNT e SUM em Bruto!K:V	Pneumonia · Calcular: TemNumero e List.Sum	2.580 vazios preservados. Em cópia de teste, a linha nova somou 12 internações.
Total de Bruto!W e validação da soma	Pneumonia · Calcular: totalFonte, validacao	2.580 SEM DADO na origem. No teste com total 12, o resultado foi OK.
Taxa de internação por 100 mil	Pneumonia · Calcular: taxa	A origem não permitiu testar a taxa com valores reais. Com 12 internações e 6.000 habitantes, a atualização retornou 200 por 100 mil.
ID, cadastro e checagens	Pneumonia · JuncaoCadastro, JuncaoReferencia, Calcular	Identificadores, nomes e checagens iguais aos resultados salvos.

A tabela de saída é carregada pela consulta. Nenhuma coluna calculada depende de fórmulas do Excel no arquivo automatizado.

Requisitos e pendências

Situação verificada no Excel entregue, com adaptação do exemplo CaféData ao tema Vila Viva.

Requisito	Situação	Evidência ou pendência
Conexão às fontes	Aplicado	Três arquivos .xlsx e três parâmetros de caminho. O Bruto da própria doença é a fonte principal.
Nulos e padronização	Aplicado	Tipos e localidade pt-BR. Limpeza de nomes, chaves municipais e tratamento de vazios e divisões conforme as regras por doença.
Remoção de duplicatas	Parcial	Unicas testa a chave e interrompe a consulta quando encontra duplicata. O código não descarta nem registra essas linhas separadamente.
Unpivot	Pendente	As saídas atuais são anuais. A visão mensal do tema precisa transformar os meses em linhas. Pneumonia -1 ainda tem os meses vazios.
Anexar as três bases	Fora do fluxo atual	O modelo de exemplo consolida filiais com estrutura comum. Aqui cada doença tem indicadores próprios e a entrega solicitada mantém as saídas separadas.
Rejeitados e Conf_Reconciliacao	Pendente	Consultas obrigatórias no guia prático. A Validação existente compara com o Tratado, mas não comprova lidas = aceitas + rejeitadas a cada atualização.

Relacionamentos e medidas DAX pertencem à Etapa 3. O Excel atual entrega tabelas de planilha para a etapa de tratamento.

Comparação com os originais

Foram comparadas as três saídas carregadas no Excel com os valores salvos nas abas Tratado dos mesmos arquivos -1.

Base	Linhas	Colunas	Diferenças	Escopo
COVID-19	2.580	28	0	Casos, óbitos, indicadores e checagens.
Dengue	2.580	27	0	Notificações, confirmados, proporções e incidência.
Pneumonia	2.580	22	0	A taxa e os valores ausentes permaneceram vazios.

Conferência por Ano + Codigo_Municipio_CVE. Foram 7.740 registros considerando as três saídas. A ordem das linhas mudou, com os mesmos registros e valores. Célula vazia e texto vazio foram tratados como equivalentes. A tolerância numérica foi de 10^{-12}, absoluta ou relativa.

Teste de atualização

Em cópias das três origens, uma nova linha por doença foi carregada com Dados → Atualizar Tudo. As três tabelas cresceram e os cálculos de teste foram conferidos.

Limites da validação

Os números oficiais não foram revalidados. Faltam dados reais mensais e populacionais de Pneumonia -1. Combinações de células com erro, tipos inválidos e tratamento de duplicatas não foram testadas exaustivamente.

Configurar e atualizar

O arquivo inclui os caminhos iniciais e a aba Instruções. A atualização lê as versões salvas das origens.

1. Configurar

Em Dados → Consultas e Conexões, abra uma consulta. No Editor do Power Query, vá a Página Inicial → Gerenciar Parâmetros.

2. Salvar as origens

Ajuste os três caminhos se necessário e use Fechar e Carregar. Nas origens, preencha Bruto, mantenha os cabeçalhos e salve os arquivos.

3. Atualizar

No arquivo automatizado, use Dados → Atualizar Tudo. Aguarde a conclusão em Consultas e Conexões e confira a tabela da doença.

Parâmetro	Arquivo esperado	Destino confirmado
pArquivoCOVID19	Vila_Viva_COVID19-1.xlsx	COVID19!A4 · tbCOVID19
pArquivoDengue	Vila_Viva_Dengue-1.xlsx	Dengue!A4 · tbDengue
pArquivoPneumonia	Vila_Viva_Pneumonia-1.xlsx	Pneumonia!A4 · tbPneumonia

Mantenha Cadastros e Tratado nos arquivos de origem. Para novas chaves de dengue e pneumonia, use a coluna opcional Populacao_Residente em Bruto ou informe a população no Tratado de origem com a mesma chave.

Artefatos e responsabilidades

O trabalho de tratamento no Excel fornece a base que a automação relê e confere.

Item	Registro para a entrega
Pasta da etapa	MA2026-entrega-parcial/02-powerquery/
Artefato verificado	Vila_Viva_02_PowerQuery.xlsx · três consultas, três parâmetros e três tabelas de saída.
Scripts e dados tratados	Código M incorporado e saídas carregadas confirmados. Exportação dos scripts finais, CSVs UTF-8 e publicação no repositório não verificadas.
Pacote de documentação	Este PDF compõe MA2026-entrega-parcial/04-documentacao/. README, capturas de evidência e commits do pacote da Etapa 2 não verificados.

Atividade da Parcial 1	Integrante	Evidência de atribuição
Tratamento das bases no Excel	Não informado	Planilhas permitem identificar o trabalho, mas não atribuir autoria individual.
Automação e validação das consultas	Não informado	Confirmação da equipe e histórico de commits pendentes.

As contribuições individuais devem ser preenchidas pela equipe com base nas tarefas realizadas e nos commits correspondentes.

Fontes da documentação

Escopo de leitura, referência acadêmica e evidências da equipe.

Arquivo	Uso neste documento
FATECSP_MA_Projeto_Orientações.pdf	Guia de Orientação e Evolução do Projeto Evolutivo de Dados. Estrutura do GitHub, Parcial 1 e responsabilidades.
FatecSP_MA_Projeto_Modelo_CafeData_V1.pdf	Projeto Modelo CaféData, seção 6.1 para Excel Avançado e 6.2 para Power Query.
FatecSP_MA_Guia_Etapa2_PowerQuery_CafeData.pdf	Guia prático de Power Query. Requisitos, carga, evidências, reconciliação e adaptação ao tema nas páginas 4-5, 36-37 e 42 do PDF.
Documentacao_Programa_Vila_Viva_Saude.html.pdf	Referência visual. Capa azul-marinho, páginas horizontais, ícones e cores das três doenças.
Arquivos -1.xlsx da equipe	Bruto, Tratado, Cadastros e Auditoria das três bases, conforme mapa de arquivos.
Vila_Viva_02_PowerQuery.xlsx	Consultas e parâmetros incorporados, tabelas de saída, Mapeamento e Validação.

COVID-19 · 1/6

pArquivoCOVID19 e COVID19 | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
section Section1;

shared pArquivoCOVID19 = "C:\Users\Estrela\Downloads\Vila_Viva_COVID19-1.xlsx" meta [IsParameterQuery = true, Type = "Text", IsParameterQueryRequired
= true]
;

shared COVID19 = let
    // O caminho é configurado em Gerenciar Parâmetros.
    Caminho = pArquivoCOVID19,
    Arquivo = Excel.Workbook(File.Contents(Caminho), [DelayTypes = true, InferSheetDimensions = true]),
    Aba = (nome as text) as table =>
        let
            candidatas = Table.SelectRows(Arquivo, each [Kind] = "Sheet" and Comparer.OrdinalIgnoreCase([Item], nome) = 0)
        in
            if Table.IsEmpty(candidatas) then error ("Aba ausente: " & nome) else candidatas{0}[Data],
    Cabecalhos = (dados as table) as table =>
        Table.PromoteHeaders(dados, [PromoteAllScalars = true, Culture = "pt-BR"]),

    Numero = (valor as any) as nullable number =>
        if valor = null or (Value.Is(valor, type text) and Text.Trim(valor) = "")
        then null
        else Number.From(valor, "pt-BR"),
    TextoLimpo = (valor as any) as nullable text =>
        if valor = null then null
        else Text.Combine(
            List.Select(
                Text.Split(Text.Clean(Text.Replace(Text.From(valor, "pt-BR"), Character.FromNumber(160), " ")), " "),
                each _ <> ""
            ),
            " "
        ),
    Codigo = (valor as any, tamanho as number) as nullable text =>
        if valor = null then null else Text.PadStart(Text.From(valor, "pt-BR"), tamanho, "0"),
    Meses = {"Janeiro", "Fevereiro", "Março", "Abril", "Maio", "Junho", "Julho", "Agosto", "Setembro", "Outubro", "Novembro", "Dezembro"},
    ValoresMensais = (linha as record, sufixo as text) as list =>
```

COVID-19 · 2/6

pArquivoCOVID19 e COVID19 | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
    List.Transform(Meses, each Record.Field(linha, _ & "_" & sufixo)),
TemNumero = (valores as list) as logical =>
    List.AnyTrue(List.Transform(valores, each try Value.Is(_, type number) otherwise false)),
Aritmetica = (valor as any) as number =>
    if valor = null then 0 else Number.From(valor, "pt-BR"),
SomaDireta = (valores as list) as number =>
    List.Accumulate(valores, 0, (total, valor) => total + Aritmetica(valor)),
Vazio = (valor as any) as logical => valor = null or valor = "",
ReferenciaDireta = (valor as any) as any => if valor = null then 0 else valor,
Razao = (numerador as nullable number, denominador as nullable number, fator as number, vazio as nullable number) as nullable number =>
    if numerador = null or denominador = null or denominador = 0 then vazio
    else numerador / denominador * fator,

Chaves = {"Ano", "Codigo_Municipio_CVE"},
TiparChaves = (dados as table) as table =>
    Table.TransformColumnTypes(dados, {{"Ano", Int64.Type}, {"Codigo_Municipio_CVE", Int64.Type}}, "pt-BR"),
SoRegistros = (dados as table) as table =>
    Table.SelectRows(dados, each [Ano] <> null and [Codigo_Municipio_CVE] <> null),
Unicas = (dados as table, colunas as list, origem as text) as table =>
    let
        mem = Table.Buffer(dados),
        duplicadas = Table.RowCount(Table.Distinct(Table.SelectColumns(mem, colunas))) <> Table.RowCount(mem)
    in
        if duplicadas then error ("Há chaves duplicadas em " & origem & ". Confira os registros antes de atualizar.")
        else mem,

Bruto = Unicas(SoRegistros(TiparChaves(Cabecalhos(Aba("Bruto")))), Chaves, "Bruto"),
CadastroSemCabecalho = Table.Skip(Aba("Cadastros"), each [Column1] <> "Codigo_Municipio_CVE"),
CadastroTipado = Table.TransformColumnTypes(
    Table.SelectRows(Cabecalhos(CadastroSemCabecalho),
        each (try Number.From([Codigo_Municipio_CVE], "pt-BR") otherwise null) <> null),
    {{"Codigo_Municipio_CVE", Int64.Type}, {"Codigo_IBGE_7_Digitos", type text}},
    "pt-BR"
),
CadastroMunicipal = Unicas(CadastroTipado, {"Codigo_Municipio_CVE"}, "Cadastros"),
```

COVID-19 · 3/6

pArquivoCOVID19 e COVID19 | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
    JuncaoCadastro = Table.NestedJoin(Bruto, {"Codigo_Municipio_CVE"}, CadastroMunicipal, {"Codigo_Municipio_CVE"}, "_Cadastro", JoinKind.LeftOuter),
    ComCadastro = Table.ExpandTableColumn(JuncaoCadastro, "_Cadastro", {"Codigo_IBGE_7_Digitos", "Municipio_Referencia"}, {"_IBGE", "_Municipio"}),
    // Lê somente os campos de referência da aba antiga. Os cálculos vêm do Bruto.
    CamposReferencia = {"ID_Registro", "Fonte", "Status_IBGE", "Data_Extracao", "Codigo_IBGE_7_Digitos", "Municipio_IBGE_2025"},
    Referencia = Unicas(
        SoRegistros(TiparChaves(Table.SelectColumns(Cabecalhos(Aba("Tratado")), Chaves & CamposReferencia))),
        Chaves,
        "referência da aba Tratado"
    ),
    JuncaoReferencia = Table.NestedJoin(ComCadastro, Chaves, Referencia, Chaves, "_Referencia", JoinKind.LeftOuter),
    Dados = Table.ExpandTableColumn(JuncaoReferencia, "_Referencia", CamposReferencia, List.Transform(CamposReferencia, each "_" & _)),
    Calcular = (r as record) as record =>
        let
            ano = Text.From(r[Ano], "pt-BR"),
            cve = Codigo(r[Codigo_Municipio_CVE], 6),
            ibge = Codigo(r[_IBGE], 7),
            municipio = r[_Municipio],
            ibgeOriginal = Codigo(r[_Codigo_IBGE_7_Digitos] ?? ibge, 7),
            municipioOriginal = r[_Municipio_IBGE_2025] ?? municipio,
            pop = Numero(r[Populacao_Fonte]),
            checagemCodigo = if Vazio(ibgeOriginal) or Vazio(ibge) then "SEM CORRESPONDÊNCIA" else if ibgeOriginal = ibge then "OK" else "DIVERGENTE",
            checagemNome = if Vazio(municipio) then "SEM CADASTRO" else if Text.Upper(TextoLimpo(municipioOriginal), "pt-BR") =
Text.Upper(TextoLimpo(municipio), "pt-BR") then "OK" else "VERIFICAR NOME",
            mesesCasos = ValoresMensais(r, "Casos_Novos"),
            mesesObitos = ValoresMensais(r, "Obitos_Novos"),
            semDados = not TemNumero(mesesCasos & mesesObitos),
            casos = if semDados then null else SomaDireta(mesesCasos),
            obitos = if semDados then null else SomaDireta(mesesObitos),
            casosFonte = Aritmetica(r[Total_Casos_Novos_Fonte]),
            obitosFonte = Aritmetica(r[Total_Obitos_Novos_Fonte]),
            incidencia = Razao(casos, pop, 100000, null),
            mortalidade = Razao(obitos, pop, 100000, null),
            letalidade = Razao(obitos, casos, 100, null),
            incidenciaFonte = Razao(casosFonte, pop, 100000, null),
            mortalidadeFonte = Razao(obitosFonte, pop, 100000, null),
```

COVID-19 · 4/6

pArquivoCOVID19 e COVID19 | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
            checagemTaxasFonte =
                if pop = null or pop <= 0 then "NÃO CALCULÁVEL"
                else if incidenciaFonte <> null and mortalidadeFonte <> null
                    and Number.Abs((incidencia ?? 0) - incidenciaFonte) < 0.000001
                    and Number.Abs((mortalidade ?? 0) - mortalidadeFonte) < 0.000001 then "OK"
                else "DIVERGENTE",
            checagemObitosFonte = if obitos <> null and obitos = ReferenciaDireta(r[Total_Obitos_Novos_Fonte]) then "OK" else "DIVERGENTE",
            checagemManual = if semDados then "AGUARDANDO DADOS" else "VERIFICAR"
        in
            [
                ID_Registro = r[_ID_Registro] ?? ("COVID-" & ano & "-" & cve),
                Ano = r[Ano],
                Codigo_Municipio_CVE = r[Codigo_Municipio_CVE],
                Municipio_Residencia = r[Municipio_Residencia],
                Casos_Novos_Ano = casos,
                Obitos_Novos_Ano = obitos,
                Casos_Acumulados_Fonte = checagemTaxasFonte,
                Obitos_Acumulados_Fonte = checagemObitosFonte,
                Validacao_Casos = if semDados then "AGUARDANDO DADOS" else if casos >= 0 and obitos >= 0 then "OK" else "VERIFICAR",
                Codigo_IBGE_7_Digitos = ibgeOriginal,
                Status_IBGE = r[_Status_IBGE] ?? (if ibge = null then "SEM CORRESPONDÊNCIA" else "Validado — cadastro municipal de referência"),
                Fonte = r[_Fonte] ?? "Fundação Seade — COVID-19 diário/semanal",
                Data_Extracao = r[_Data_Extracao] ?? r[Data_Extracao],
                Municipio_IBGE_2025 = municipioOriginal,
                Populacao_Residente = pop,
                Incidencia_Casos_100mil = incidencia,
                Mortalidade_Obitos_100mil = mortalidade,
                Letalidade_pct = letalidade,
                Municipio_Normalizado = TextoLimpo(r[Municipio_Residencia]),
                Codigo_IBGE_Cadastro = ibge,
                Municipio_Cadastro = municipio,
                Checagem_Codigo_IBGE = checagemCodigo,
                Checagem_Nome_Municipio = checagemNome,
                Checagem_Casos = checagemManual,
                Checagem_Obitos = checagemManual,
```

COVID-19 · 5/6

pArquivoCOVID19 e COVID19 | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
                Checagem_Populacao = if pop <> null and pop > 0 then "OK" else "AUSENTE/INVÁLIDA",
                Checagem_Indicadores = if incidencia = null or mortalidade = null or letalidade = null then "NÃO CALCULÁVEL" else "CALCULADO",
                Status_Dados = if semDados then "AGUARDANDO BASE OFICIAL" else "DADOS A CONFERIR"
            ],

    Colunas = {
        "ID_Registro",
        "Ano",
        "Codigo_Municipio_CVE",
        "Municipio_Residencia",
        "Casos_Novos_Ano",
        "Obitos_Novos_Ano",
        "Casos_Acumulados_Fonte",
        "Obitos_Acumulados_Fonte",
        "Validacao_Casos",
        "Codigo_IBGE_7_Digitos",
        "Status_IBGE",
        "Fonte",
        "Data_Extracao",
        "Municipio_IBGE_2025",
        "Populacao_Residente",
        "Incidencia_Casos_100mil",
        "Mortalidade_Obitos_100mil",
        "Letalidade_pct",
        "Municipio_Normalizado",
        "Codigo_IBGE_Cadastro",
        "Municipio_Cadastro",
        "Checagem_Codigo_IBGE",
        "Checagem_Nome_Municipio",
        "Checagem_Casos",
        "Checagem_Obitos",
        "Checagem_Populacao",
        "Checagem_Indicadores",
        "Status_Dados"
    },
```

COVID-19 · 6/6

pArquivoCOVID19 e COVID19 | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
    Calculadas = Table.FromRecords(List.Transform(Table.ToRecords(Dados), Calcular), Colunas),
    ComTipos = Table.TransformColumnTypes(Calculadas, {
        {"ID_Registro", type text},
        {"Ano", Int64.Type},
        {"Codigo_Municipio_CVE", Int64.Type},
        {"Municipio_Residencia", type text},
        {"Casos_Novos_Ano", type number},
        {"Obitos_Novos_Ano", type number},
        {"Casos_Acumulados_Fonte", type text},
        {"Obitos_Acumulados_Fonte", type text},
        {"Validacao_Casos", type text},
        {"Codigo_IBGE_7_Digitos", type text},
        {"Status_IBGE", type text},
        {"Fonte", type text},
        {"Municipio_IBGE_2025", type text},
        {"Populacao_Residente", type number},
        {"Incidencia_Casos_100mil", type number},
        {"Mortalidade_Obitos_100mil", type number},
        {"Letalidade_pct", type number},
        {"Municipio_Normalizado", type text},
        {"Codigo_IBGE_Cadastro", type text},
        {"Municipio_Cadastro", type text},
        {"Checagem_Codigo_IBGE", type text},
        {"Checagem_Nome_Municipio", type text},
        {"Checagem_Casos", type text},
        {"Checagem_Obitos", type text},
        {"Checagem_Populacao", type text},
        {"Checagem_Indicadores", type text},
        {"Status_Dados", type text}
    }, "pt-BR")
in
    ComTipos
;
```

Dengue · 1/6

pArquivoDengue e Dengue | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
shared pArquivoDengue = "C:\Users\Estrela\Downloads\Vila_Viva_Dengue-1.xlsx" meta [IsParameterQuery = true, Type = "Text", IsParameterQueryRequired = true]
;

shared Dengue = let
    // O caminho é configurado em Gerenciar Parâmetros.
    Caminho = pArquivoDengue,
    Arquivo = Excel.Workbook(File.Contents(Caminho), [DelayTypes = true, InferSheetDimensions = true]),
    Aba = (nome as text) as table =>
        let
            candidatas = Table.SelectRows(Arquivo, each [Kind] = "Sheet" and Comparer.OrdinalIgnoreCase([Item], nome) = 0)
        in
            if Table.IsEmpty(candidatas) then error ("Aba ausente: " & nome) else candidatas{0}[Data],
    Cabecalhos = (dados as table) as table =>
        Table.PromoteHeaders(dados, [PromoteAllScalars = true, Culture = "pt-BR"]),

    Numero = (valor as any) as nullable number =>
        if valor = null or (Value.Is(valor, type text) and Text.Trim(valor) = "")
        then null
        else Number.From(valor, "pt-BR"),
    TextoLimpo = (valor as any) as nullable text =>
        if valor = null then null
        else Text.Combine(
            List.Select(
                Text.Split(Text.Clean(Text.Replace(Text.From(valor, "pt-BR"), Character.FromNumber(160), " ")), " "),
                each _ <> ""
            ),
            " "
        ),
    Codigo = (valor as any, tamanho as number) as nullable text =>
        if valor = null then null else Text.PadStart(Text.From(valor, "pt-BR"), tamanho, "0"),
    Meses = {"Janeiro", "Fevereiro", "Março", "Abril", "Maio", "Junho", "Julho", "Agosto", "Setembro", "Outubro", "Novembro", "Dezembro"},
    ValoresMensais = (linha as record, sufixo as text) as list =>
        List.Transform(Meses, each Record.Field(linha, _ & "_" & sufixo)),
    TemNumero = (valores as list) as logical =>
```

Dengue · 2/6

pArquivoDengue e Dengue | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
        List.AnyTrue(List.Transform(valores, each try Value.Is(_, type number) otherwise false)),
    Aritmetica = (valor as any) as number =>
        if valor = null then 0 else Number.From(valor, "pt-BR"),
    SomaDireta = (valores as list) as number =>
        List.Accumulate(valores, 0, (total, valor) => total + Aritmetica(valor)),
    Vazio = (valor as any) as logical => valor = null or valor = "",
    ReferenciaDireta = (valor as any) as any => if valor = null then 0 else valor,
    Razao = (numerador as nullable number, denominador as nullable number, fator as number, vazio as nullable number) as nullable number =>
        if numerador = null or denominador = null or denominador = 0 then vazio
        else numerador / denominador * fator,

    Chaves = {"Ano", "Codigo_Municipio_CVE"},
    TiparChaves = (dados as table) as table =>
        Table.TransformColumnTypes(dados, {{"Ano", Int64.Type}, {"Codigo_Municipio_CVE", Int64.Type}}, "pt-BR"),
    SoRegistros = (dados as table) as table =>
        Table.SelectRows(dados, each [Ano] <> null and [Codigo_Municipio_CVE] <> null),
    Unicas = (dados as table, colunas as list, origem as text) as table =>
        let
            mem = Table.Buffer(dados),
            duplicadas = Table.RowCount(Table.Distinct(Table.SelectColumns(mem, colunas))) <> Table.RowCount(mem)
        in
            if duplicadas then error ("Há chaves duplicadas em " & origem & ". Confira os registros antes de atualizar.")
            else mem,

    Bruto = Unicas(SoRegistros(TiparChaves(Cabecalhos(Aba("Bruto")))), Chaves, "Bruto"),
    CadastroSemCabecalho = Table.Skip(Aba("Cadastros"), each [Column1] <> "Codigo_Municipio_CVE"),
    CadastroTipado = Table.TransformColumnTypes(
        Table.SelectRows(Cabecalhos(CadastroSemCabecalho),
            each (try Number.From([Codigo_Municipio_CVE], "pt-BR") otherwise null) <> null),
        {{"Codigo_Municipio_CVE", Int64.Type}, {"Codigo_IBGE_7_Digitos", type text}},
        "pt-BR"
    ),
    CadastroMunicipal = Unicas(CadastroTipado, {"Codigo_Municipio_CVE"}, "Cadastros"),
    JuncaoCadastro = Table.NestedJoin(Bruto, {"Codigo_Municipio_CVE"}, CadastroMunicipal, {"Codigo_Municipio_CVE"}, "_Cadastro", JoinKind.LeftOuter),
    ComCadastro = Table.ExpandTableColumn(JuncaoCadastro, "_Cadastro", {"Codigo_IBGE_7_Digitos", "Municipio_Referencia"}, {"_IBGE", "_Municipio"}),
```

Dengue · 3/6

pArquivoDengue e Dengue | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
70          // Lê somente os campos de referência da aba antiga. Os cálculos vêm do Bruto.
71          CamposReferencia = {"ID_Registro", "Fonte", "Status_IBGE", "Data_Extracao", "Codigo_IBGE_7_Digitos", "Municipio_IBGE_2025",
        "Populacao_Residente"},
72          Referencia = Unicas(
73              SoRegistros(TiparChaves(Table.SelectColumns(Cabecalhos(Aba("Tratado")), Chaves & CamposReferencia))),
74              Chaves,
75              "referência da aba Tratado"
76          ),
77          JuncaoReferencia = Table.NestedJoin(ComCadastro, Chaves, Referencia, Chaves, "_Referencia", JoinKind.LeftOuter),
78          Dados = Table.ExpandTableColumn(JuncaoReferencia, "_Referencia", CamposReferencia, List.Transform(CamposReferencia, each "_" & _)),
79          Calcular = (r as record) as record =>
80              let
81                  ano = Text.From(r[Ano], "pt-BR"),
82                  cve = Codigo(r[Codigo_Municipio_CVE], 6),
83                  ibge = Codigo(r[_IBGE], 7),
84                  municipio = r[_Municipio],
85                  ibgeOriginal = Codigo(r[_Codigo_IBGE_7_Digitos] ?? ibge, 7),
86                  municipioOriginal = r[_Municipio_IBGE_2025] ?? municipio,
87                  pop = Numero(Record.FieldOrDefault(r, "Populacao_Residente", null)) ?? Numero(r[_Populacao_Residente]),
88                  checagemCodigo = if Vazio(ibgeOriginal) or Vazio(ibge) then "SEM CORRESPONDÊNCIA" else if ibgeOriginal = ibge then "OK" else "DIVERGENTE",
89                  checagemNome = if Vazio(municipio) then "SEM CADASTRO" else if Text.Upper(TextoLimpo(municipioOriginal), "pt-BR") =
        Text.Upper(TextoLimpo(municipio), "pt-BR") then "OK" else "VERIFICAR NOME",
90                  auto = ReferenciaDireta(r[Total_Autoctones_Fonte]),
91                  imp = ReferenciaDireta(r[Total_Importados_Fonte]),
92                  total = Aritmetica(auto) + Aritmetica(imp),
93                  notificados = SomaDireta(ValoresMensais(r, "Notificados")),
94                  incidencia = try Razao(total, pop, 100000, 0) otherwise 0
95              in
96                  [
97                      ID_Registro = r[_ID_Registro] ?? ("DENGUE-" & ano & "-" & cve),
98                      Ano = r[Ano],
99                      Codigo_Municipio_CVE = r[Codigo_Municipio_CVE],
100                     Municipio_Residencia = r[Municipio_Residencia],
101                     Confirmados_Autoctones = auto,
102                     Confirmados_Importados = imp,
```

Dengue · 4/6

pArquivoDengue e Dengue | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
                Confirmados_Total = total,
                Total_Confirmados_Fonte = total,
                Validacao_Soma = if total = Aritmetica(r[Total_Autoctones_Fonte]) + Aritmetica(r[Total_Importados_Fonte]) then "OK" else "DIVERGENTE",
                Codigo_IBGE_7_Digitos = ibgeOriginal,
                Status_IBGE = r[_Status_IBGE] ?? (if ibge = null then "SEM CORRESPONDÊNCIA" else "Validado — DTB IBGE 2025"),
                Fonte = r[_Fonte] ?? ("CVE-SP — tabela mensal " & ano),
                Data_Extracao = r[_Data_Extracao] ?? r[Data_Extracao],
                Municipio_IBGE_2025 = municipioOriginal,
                Notificados_Anual_Calculado = notificados,
                Proporcao_Autoctones_pct = try Razao(Aritmetica(auto), total, 1, 0) otherwise 0,
                Proporcao_Importados_pct = try Razao(Aritmetica(imp), total, 1, 0) otherwise 0,
                Populacao_Residente = pop,
                Incidencia_Confirmados_100mil = incidencia,
                Municipio_Normalizado = TextoLimpo(r[Municipio_Residencia]),
                Codigo_IBGE_Cadastro = ibge,
                Municipio_Cadastro = municipio,
                Checagem_Codigo_IBGE = checagemCodigo,
                Checagem_Nome_Municipio = checagemNome,
                Checagem_Confirmados = if total = Aritmetica(auto) + Aritmetica(imp) then "OK" else "DIVERGENTE",
                Checagem_Populacao = if pop <> null and pop > 0 then "OK" else "AUSENTE/INVÁLIDA",
                Checagem_Incidencia = if pop = null or pop <= 0 then "NÃO CALCULÁVEL" else if Number.Abs(incidencia - total / pop * 100000) <
0.000001 then "OK" else "DIVERGENTE"
            ],

    Colunas = {
        "ID_Registro",
        "Ano",
        "Codigo_Municipio_CVE",
        "Municipio_Residencia",
        "Confirmados_Autoctones",
        "Confirmados_Importados",
        "Confirmados_Total",
        "Total_Confirmados_Fonte",
        "Validacao_Soma",
        "Codigo_IBGE_7_Digitos",
```

Dengue · 5/6

pArquivoDengue e Dengue | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
            "Status_IBGE",
            "Fonte",
            "Data_Extracao",
            "Municipio_IBGE_2025",
            "Notificados_Anual_Calculado",
            "Proporcao_Autoctones_pct",
            "Proporcao_Importados_pct",
            "Populacao_Residente",
            "Incidencia_Confirmados_100mil",
            "Municipio_Normalizado",
            "Codigo_IBGE_Cadastro",
            "Municipio_Cadastro",
            "Checagem_Codigo_IBGE",
            "Checagem_Nome_Municipio",
            "Checagem_Confirmados",
            "Checagem_Populacao",
            "Checagem_Incidencia"
        },
        Calculadas = Table.FromRecords(List.Transform(Table.ToRecords(Dados), Calcular), Colunas),
        ComTipos = Table.TransformColumnTypes(Calculadas, {
            {"ID_Registro", type text},
            {"Ano", Int64.Type},
            {"Codigo_Municipio_CVE", Int64.Type},
            {"Municipio_Residencia", type text},
            {"Confirmados_Total", type number},
            {"Total_Confirmados_Fonte", type number},
            {"Validacao_Soma", type text},
            {"Codigo_IBGE_7_Digitos", type text},
            {"Status_IBGE", type text},
            {"Fonte", type text},
            {"Municipio_IBGE_2025", type text},
            {"Notificados_Anual_Calculado", type number},
            {"Proporcao_Autoctones_pct", type number},
            {"Proporcao_Importados_pct", type number},
            {"Populacao_Residente", type number},
```

Dengue · 6/6

pArquivoDengue e Dengue | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
            {"Incidencia_Confirmados_100mil", type number},
            {"Municipio_Normalizado", type text},
            {"Codigo_IBGE_Cadastro", type text},
            {"Municipio_Cadastro", type text},
            {"Checagem_Codigo_IBGE", type text},
            {"Checagem_Nome_Municipio", type text},
            {"Checagem_Confirmados", type text},
            {"Checagem_Populacao", type text},
            {"Checagem_Incidencia", type text}
        }, "pt-BR")
    in
        ComTipos
    ;
```

Pneumonia · 1/5

pArquivoPneumonia e Pneumonia | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
shared pArquivoPneumonia = "C:\Users\Estrela\Downloads\Vila_Viva_Pneumonia-1.xlsx" meta [IsParameterQuery = true, Type = "Text",
IsParameterQueryRequired = true]
;

shared Pneumonia = let
    // O caminho é configurado em Gerenciar Parâmetros.
    Caminho = pArquivoPneumonia,
    Arquivo = Excel.Workbook(File.Contents(Caminho), [DelayTypes = true, InferSheetDimensions = true]),
    Aba = (nome as text) as table =>
        let
            candidatas = Table.SelectRows(Arquivo, each [Kind] = "Sheet" and Comparer.OrdinalIgnoreCase([Item], nome) = 0)
        in
            if Table.IsEmpty(candidatas) then error ("Aba ausente: " & nome) else candidatas{0}[Data],
    Cabecalhos = (dados as table) as table =>
        Table.PromoteHeaders(dados, [PromoteAllScalars = true, Culture = "pt-BR"]),

    Numero = (valor as any) as nullable number =>
        if valor = null or (Value.Is(valor, type text) and Text.Trim(valor) = "")
        then null
        else Number.From(valor, "pt-BR"),
    TextoLimpo = (valor as any) as nullable text =>
        if valor = null then null
        else Text.Combine(
            List.Select(
                Text.Split(Text.Clean(Text.Replace(Text.From(valor, "pt-BR"), Character.FromNumber(160), " ")), " "),
                each _ <> ""
            ),
            " "
        ),
    Codigo = (valor as any, tamanho as number) as nullable text =>
        if valor = null then null else Text.PadStart(Text.From(valor, "pt-BR"), tamanho, "0"),
    Meses = {"Janeiro", "Fevereiro", "Março", "Abril", "Maio", "Junho", "Julho", "Agosto", "Setembro", "Outubro", "Novembro", "Dezembro"},
    ValoresMensais = (linha as record, sufixo as text) as list =>
        List.Transform(Meses, each Record.Field(linha, _ & "_" & sufixo)),
    TemNumero = (valores as list) as logical =>
```

Pneumonia · 2/5

pArquivoPneumonia e Pneumonia | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
        List.AnyTrue(List.Transform(valores, each try Value.Is(_, type number) otherwise false)),
    Aritmetica = (valor as any) as number =>
        if valor = null then 0 else Number.From(valor, "pt-BR"),
    SomaDireta = (valores as list) as number =>
        List.Accumulate(valores, 0, (total, valor) => total + Aritmetica(valor)),
    Vazio = (valor as any) as logical => valor = null or valor = "",
    ReferenciaDireta = (valor as any) as any => if valor = null then 0 else valor,
    Razao = (numerador as nullable number, denominador as nullable number, fator as number, vazio as nullable number) as nullable number =>
        if numerador = null or denominador = null or denominador = 0 then vazio
        else numerador / denominador * fator,

    Chaves = {"Ano", "Codigo_Municipio_CVE"},
    TiparChaves = (dados as table) as table =>
        Table.TransformColumnTypes(dados, {{"Ano", Int64.Type}, {"Codigo_Municipio_CVE", Int64.Type}}, "pt-BR"),
    SoRegistros = (dados as table) as table =>
        Table.SelectRows(dados, each [Ano] <> null and [Codigo_Municipio_CVE] <> null),
    Unicas = (dados as table, colunas as list, origem as text) as table =>
        let
            mem = Table.Buffer(dados),
            duplicadas = Table.RowCount(Table.Distinct(Table.SelectColumns(mem, colunas))) <> Table.RowCount(mem)
        in
            if duplicadas then error ("Há chaves duplicadas em " & origem & ". Confira os registros antes de atualizar.")
            else mem,

    Bruto = Unicas(SoRegistros(TiparChaves(Cabecalhos(Aba("Bruto")))), Chaves, "Bruto"),
    CadastroSemCabecalho = Table.Skip(Aba("Cadastros"), each [Column1] <> "Codigo_Municipio_CVE"),
    CadastroTipado = Table.TransformColumnTypes(
        Table.SelectRows(Cabecalhos(CadastroSemCabecalho),
            each (try Number.From([Codigo_Municipio_CVE], "pt-BR") otherwise null) <> null),
        {{"Codigo_Municipio_CVE", Int64.Type}, {"Codigo_IBGE_7_Digitos", type text}},
        "pt-BR"
    ),
    CadastroMunicipal = Unicas(CadastroTipado, {"Codigo_Municipio_CVE"}, "Cadastros"),
    JuncaoCadastro = Table.NestedJoin(Bruto, {"Codigo_Municipio_CVE"}, CadastroMunicipal, {"Codigo_Municipio_CVE"}, "_Cadastro", JoinKind.LeftOuter),
    ComCadastro = Table.ExpandTableColumn(JuncaoCadastro, "_Cadastro", {"Codigo_IBGE_7_Digitos", "Municipio_Referencia"}, {"_IBGE", "_Municipio"}),
```

Pneumonia · 3/5

pArquivoPneumonia e Pneumonia | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
        // Lê somente os campos de referência da aba antiga. Os cálculos vêm do Bruto.
        CamposReferencia = {"Populacao_Residente"},
        Referencia = Unicas(
            SoRegistros(TiparChaves(Table.SelectColumns(Cabecalhos(Aba("Tratado")), Chaves & CamposReferencia))),
            Chaves,
            "referência da aba Tratado"
        ),
        JuncaoReferencia = Table.NestedJoin(ComCadastro, Chaves, Referencia, Chaves, "_Referencia", JoinKind.LeftOuter),
        Dados = Table.ExpandTableColumn(JuncaoReferencia, "_Referencia", CamposReferencia, List.Transform(CamposReferencia, each "_" & _)),
        Calcular = (r as record) as record =>
            let
                ano = Text.From(r[Ano], "pt-BR"),
                cve = Codigo(r[Codigo_Municipio_CVE], 6),
                ibge = Codigo(r[_IBGE], 7),
                municipio = r[_Municipio],
                pop = Numero(Record.FieldOrDefault(r, "Populacao_Residente", null)) ?? Numero(r[_Populacao_Residente]),
                checagemCodigo = if ibge = null then "SEM CORRESPONDÊNCIA" else "OK",
                checagemNome = if municipio = null then "SEM CADASTRO" else "OK",
                mensais = ValoresMensais(r, "Internacoes_Pneumonia"),
                internacoes = if not TemNumero(mensais) then null
                    else List.Sum(List.Select(mensais, each Value.Is(_, type number))),
                totalFonte = if Vazio(r[Total_Internacoes_Fonte]) then null else r[Total_Internacoes_Fonte],
                validacao = if internacoes = null then "SEM DADO"
                    else if internacoes = totalFonte then "OK" else "DIVERGENTE",
                taxa = if internacoes = null or pop = null then null
                    else if pop = 0 then error "Divisão por zero na taxa de internação"
                    else internacoes / pop * 100000
            in
                [
                    ID_Registro = "PNE-" & ano & "-" & cve,
                    Ano = r[Ano],
                    Codigo_Municipio_CVE = r[Codigo_Municipio_CVE],
                    Municipio_Residencia = r[Municipio_Residencia],
                    Internacoes_Pneumonia = internacoes,
                    Total_Internacoes_Fonte = totalFonte,
```

Pneumonia · 4/5

pArquivoPneumonia e Pneumonia | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
                Validacao_Soma = validacao,
                Codigo_IBGE_7_Digitos = ibge,
                Status_IBGE = if ibge = null then "SEM CORRESPONDÊNCIA" else "Validado — cadastro municipal",
                Fonte = "DATASUS — SIH/SUS (AIH reduzida), diagnóstico principal CID-10 J12–J18",
                Data_Extracao = if Vazio(r[Data_Extracao]) then null else r[Data_Extracao],
                Municipio_IBGE_2025 = municipio,
                Populacao_Residente = pop,
                Taxa_Internacao_100mil = taxa,
                Municipio_Normalizado = TextoLimpo(r[Municipio_Residencia]),
                Codigo_IBGE_Cadastro = ibge,
                Municipio_Cadastro = municipio,
                Checagem_Codigo_IBGE = checagemCodigo,
                Checagem_Nome_Municipio = checagemNome,
                Checagem_Internacoes = validacao,
                Checagem_Populacao = if pop = null then "NÃO INFORMADA" else "OK",
                Checagem_Taxa = if taxa = null then "NÃO CALCULÁVEL" else if Number.Abs(taxa - internacoes / pop * 100000) < 0.000001 then "OK" else
"DIVERGENTE"
            ],

    Colunas = {
        "ID_Registro",
        "Ano",
        "Codigo_Municipio_CVE",
        "Municipio_Residencia",
        "Internacoes_Pneumonia",
        "Total_Internacoes_Fonte",
        "Validacao_Soma",
        "Codigo_IBGE_7_Digitos",
        "Status_IBGE",
        "Fonte",
        "Data_Extracao",
        "Municipio_IBGE_2025",
        "Populacao_Residente",
        "Taxa_Internacao_100mil",
        "Municipio_Normalizado",
```

Pneumonia · 5/5

pArquivoPneumonia e Pneumonia | Código extraído do arquivo entregue. Numeração e quebras visuais são editoriais.

```
            "Codigo_IBGE_Cadastro",
            "Municipio_Cadastro",
            "Checagem_Codigo_IBGE",
            "Checagem_Nome_Municipio",
            "Checagem_Internacoes",
            "Checagem_Populacao",
            "Checagem_Taxa"
        },
        Calculadas = Table.FromRecords(List.Transform(Table.ToRecords(Dados), Calcular), Colunas),
        ComTipos = Table.TransformColumnTypes(Calculadas, {
            {"ID_Registro", type text},
            {"Ano", Int64.Type},
            {"Codigo_Municipio_CVE", Int64.Type},
            {"Municipio_Residencia", type text},
            {"Internacoes_Pneumonia", type number},
                    {"Validacao_Soma", type text},
            {"Codigo_IBGE_7_Digitos", type text},
            {"Status_IBGE", type text},
            {"Fonte", type text},
            {"Municipio_IBGE_2025", type text},
            {"Populacao_Residente", type number},
            {"Taxa_Internacao_100mil", type number},
            {"Municipio_Normalizado", type text},
            {"Codigo_IBGE_Cadastro", type text},
            {"Municipio_Cadastro", type text},
            {"Checagem_Codigo_IBGE", type text},
            {"Checagem_Nome_Municipio", type text},
            {"Checagem_Internacoes", type text},
            {"Checagem_Populacao", type text},
            {"Checagem_Taxa", type text}
        }, "pt-BR")
    in
        ComTipos
    ;
```